

סעיף א (10 נקודות)

חשבו את הפרמטרים השונים של הלסטד. לארבעה הראשונים יש לפרט איך הגעתם למספר זה

$\eta_1 = 12$ //	public, static, int, <=, =, ;, *=, ++, (), {}, return, for
נשים לב שאופרטור <= הוא אחד ולא שניים. זה נכון גם לגבי אופרטור ++ ו *. אמנם אפשר היה לפרק אותם לשניים ואז זה גם היה מגדיל את כמות השימוש במשתנים, אבל זה מנוגד לחשיבה של הלסטד שהסיבוכיות והסיכוי לבאגים נובעים מגודל התוכנה. זה כמובן גם מתקשר לחסרונות של השיטה.	
$\eta_2 = 5$ //	n, result, 1, i, factorial
$N_1 = 22$ //	public:1, static:1, int:4, <=:1, =:2, ;:5, *=:1, ++:1, ():2, {}:2, return:1, for:1
$N_2 = 12$ //	n:2, result:3, 1:2, i:4, factorial: 1
$\eta = 17$ //	$\eta_1 + \eta_2$
$N = 34$ //	$N_1 + N_2$
$V = 34 * \log_2(17) = 138.973736603$	
$\check{N} = 12 * \log_2(12) + 5 * \log_2(5) = 54.6291904831$	
$D = (12/2) * (12/5) = 14.4$	
$E = 14.4 * 138.973736603 = 2001.22180708$	
$T = 2001.22180708 / 18 = 111.178989282$	
$B = 138.973736603 / 3000 = 0.04632457886$	

סעיף ב (6 נקודות)

ציינו והסבירו בקצרה שתי חסרונות של שיטה זו

- No standards for a line of code
- Developed in the context of assembly languages and too fine grained for modern programming languages.
 - Highly biased by software language, developer's skills and experience
- The treatment of basic and derived measures is somehow confusing.

- The notions of time to develop and remaining bugs are arguable.
- Programming language and programmer skills are not incorporated.